

Rapport de Chef d'Œuvre

Module R506 - Sensibilisation Multimédia

Ultimate Neon Space

Diema David

Janvier 2026

Table des matières

1	Introduction	3
2	Architecture Technique	3
2.1	Choix du Langage et des Outils	3
2.2	Schéma de l'Architecture Hybride	3
2.3	Structure des Données	4
3	Implémentation Vectorielle et Animation	4
3.1	Génération Procédurale et Mathématiques	4
3.2	Le Moteur d'Export SVG (Le Pont)	5
4	Traitement Matriciel et Colorimétrie	5
4.1	Manipulation du Buffer Vidéo	5
4.2	Modèles Colorimétriques Avancés	5
5	Limites	6
5.1	Limitations Techniques	6
5.2	Conclusion	6

1 Introduction

Dans le cadre du module R506, nous explorons les bases de l'image numérique. On a tendance à penser que le vectoriel (mathématique) et le matriciel (pixels) s'opposent, mais en réalité ces deux approches se complètent bien en programmation multimédia.

Pour ce projet, je ne voulais pas simplement assembler des images existantes dans un moteur de jeu standard. J'ai préféré créer une application autonome pour manipuler ces concepts directement dans le code.

Je me suis donc fixé une contrainte technique : ne charger aucun fichier externe. Tous les éléments visuels, du vaisseau aux décors, sont générés en temps réel par des algorithmes. Cette méthode m'a obligé à reconstruire pas mal d'éléments de base. J'ai dû créer un moteur physique pour gérer les déplacements, mettre en place un système de coordonnées pour le dessin, et développer des algorithmes pour manipuler les pixels. Ce rapport explique comment j'ai fait fonctionner ensemble ces différentes parties techniques pour créer un moteur de jeu hybride.

2 Architecture Technique

2.1 Choix du Langage et des Outils

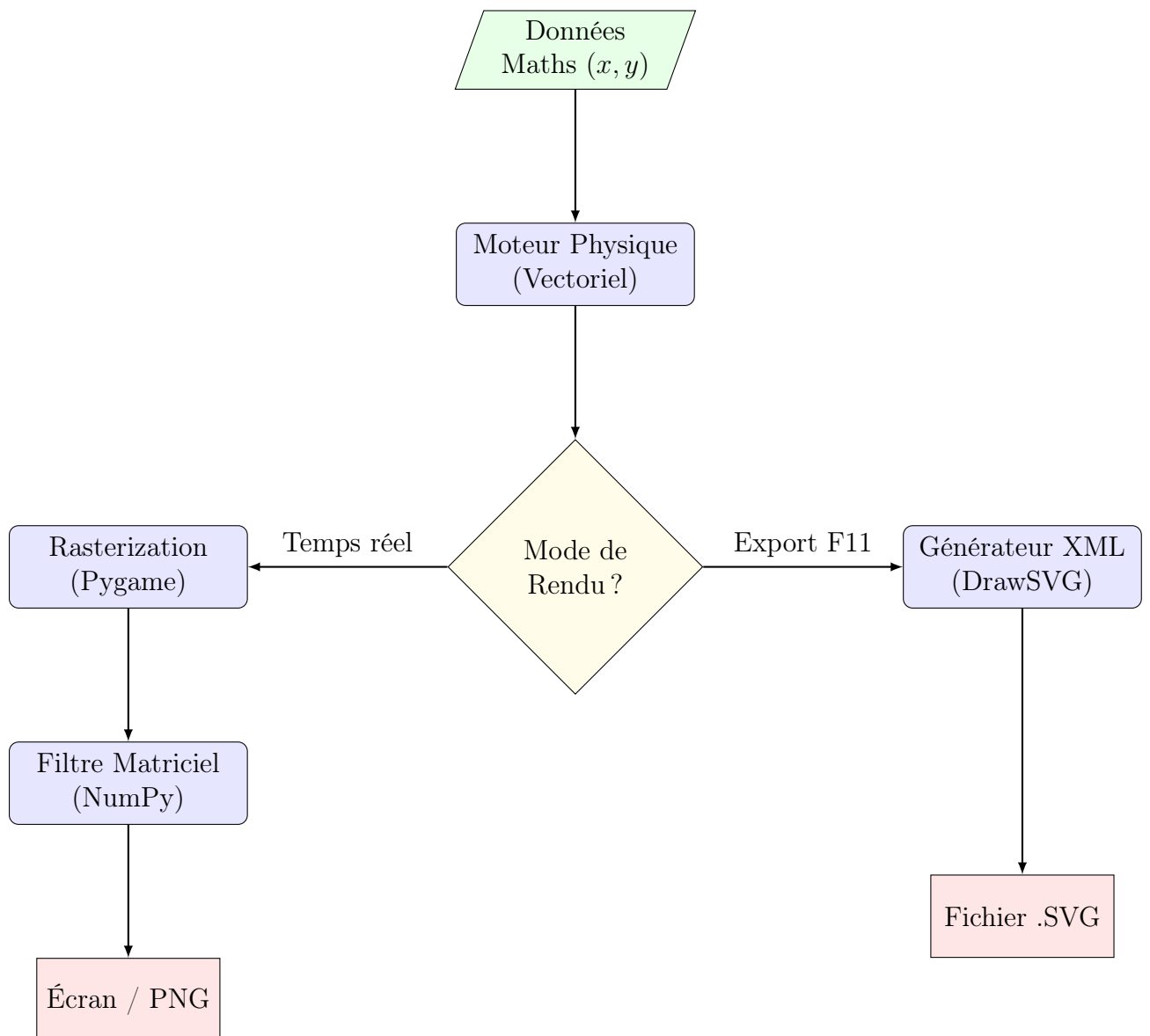
Le projet est entièrement développé en Python 3.12. J'ai choisi ce langage parce que le code est facile à lire et qu'il y a pas mal de bibliothèques utiles pour ce genre de projet.

L'architecture repose sur trois bibliothèques principales :

- **Pygame (Interface Rasterization)** : Elle gère l'affichage pixel par pixel et la détection des touches du clavier.
- **NumPy (Calcul Matriciel)** : C'est la bibliothèque la plus importante pour le traitement d'image. Elle me permet de manipuler le buffer vidéo comme une matrice en 3D, ce qui rend possible le traitement de 2 millions de pixels en temps réel.
- **DrawSVG (Génération XML)** : Je l'utilise pour l'export vectoriel. Elle transforme les objets du jeu en format SVG standard.

2.2 Schéma de l'Architecture Hybride

Voici une représentation du flux de données dans le moteur, illustrant la séparation entre le calcul logique (Vectoriel) et les sorties (Matricielles ou XML) :



2.3 Structure des Données

Contrairement à un jeu classique qui manipule des rectangles (Bounding Boxes) et des images, mon moteur manipule des listes de coordonnées. Chaque objet est une instance de classe possédant une position globale (x, y) , une liste de points locaux définissant sa forme (ex : $[(0, -25), (-20, 20), \dots]$), un angle de rotation θ et un vecteur de vitesse.

3 Implémentation Vectorielle et Animation

3.1 Génération Procédurale et Mathématiques

Au lancement, le programme ne charge aucune ressource graphique. Toutes les formes sont générées de façon géométrique. Par exemple, les astéroïdes ne sont pas dessinés à la main : un algorithme crée un polygone irrégulier en faisant varier de manière aléatoire le rayon d'un cercle sur plusieurs segments.

Pour l'animation, le problème principal était la rotation des formes. J'ai codé une fonction de transformation affine qui utilise une matrice de rotation 2D. Pour chaque point d'un objet, sa position à l'écran (x', y') est recalculée à chaque image avec la formule suivante :

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix} + \begin{pmatrix} c_x \\ c_y \end{pmatrix}$$

Cette méthode garantit que la rotation ne dégrade pas la qualité de l'image, contrairement à ce qui se passe quand on fait tourner un sprite bitmap.

3.2 Le Moteur d'Export SVG (Le Pont)

La preuve que le projet est vraiment vectoriel se trouve dans la fonctionnalité d'export (touche F11). Quand j'active cette commande, le moteur graphique se met en pause. Il parcourt toutes les listes de points en mémoire et, au lieu de les afficher à l'écran (en pixels), il génère des balises XML `<polygon>` avec les couleurs en hexadécimal. Le résultat est un fichier vectoriel léger qui décrit la scène de façon mathématique et qui peut être imprimé en très grand format sans perdre en qualité.

4 Traitement Matriciel et Colorimétrie

4.1 Manipulation du Buffer Vidéo

Le projet intègre une fonctionnalité de "Pause" (touche P). Techniquement, ça fonctionne comme ça :

1. Verrouiller la surface d'affichage de Pygame.
2. Extraire le tableau de pixels sous forme de matrice NumPy `[Width, Height, 3]`.
3. Appliquer une opération sur cette matrice.

L'opération choisie est la conversion en niveaux de gris par moyenne arithmétique :

$$Pixel_{gris} = \frac{R + G + B}{3}$$

Cette valeur est ensuite appliquée sur les trois canaux. L'utilisation de NumPy permet d'exécuter ce calcul sur environ 700 000 pixels instantanément, alors qu'une boucle Python classique aurait provoqué un gel de plusieurs secondes.

4.2 Modèles Colorimétriques Avancés

Le cours insiste sur l'importance de choisir le bon modèle de couleur.

- **HSV pour l'animation** : Pour créer l'effet "Arc-en-ciel" du vaisseau, travailler en RGB est compliqué (il faut gérer 3 courbes sinusoïdales déphasées). J'ai utilisé le modèle Teinte-Saturation-Valeur (HSV). Je fixe la Saturation et la Valeur à 100%, et je fais simplement varier la Teinte de 0° à 360° en fonction du temps.

- **Synthèse Additive (Alpha)** : Pour simuler l'aspect lumineux du néon, j'utilise le canal Alpha (transparence). Les particules ne sont pas opaques : elles sont dessinées avec une transparence élevée. Grâce au mode de fusion (Blending), quand plusieurs particules se superposent, ça finit par donner du blanc, ce qui reproduit le comportement réel de la lumière.

5 Limites

5.1 Limitations Techniques

Le choix du langage Python impose certaines limites. Étant un langage interprété, le coût CPU de la boucle de jeu augmente linéairement avec le nombre d'objets. Au-delà de 1000 particules gérant la transparence, le *framerate* peut chuter. Une optimisation possible serait de déporter les calculs de position sur le GPU (via des Shaders), mais cela sort du cadre de ce module. De plus, l'interface utilisateur est minimaliste (clavier uniquement), ce qui correspond au cahier des charges d'un prototype technique mais pourrait être amélioré pour un produit grand public.

5.2 Conclusion

Ce projet "Ultimate Neon Space" m'a permis de synthétiser de manière concrète les enseignements théoriques du module R506. J'ai pu démontrer la complémentarité des approches : le **vectoriel** pour la précision du mouvement et l'indépendance à la résolution, et le **matriciel** pour le traitement final des pixels et les effets visuels. La réalisation des convertisseurs (Export SVG et Capture PNG) a été particulièrement formatrice, car elle oblige à comprendre comment l'ordinateur encode et stocke l'information graphique sous différentes formes.